Causal Deep Learning Framework for Remaining Useful Life Estimation in Turbofan Engines

Sai Deepak Sana
cs23btech11055

Ponguru Bhanu
cs23btech11046

Peddineni Harshith
cs23btech11045

भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

A Report Submitted to Dr. Sangkha Borah
Indian Institute of Technology Hyderabad

Project Report

Abstract

Remaining Useful Life (RUL) estimation for turbofan engines is a central predictive-maintenance task because it supports maintenance decisions before failure occurs. In a realistic setting, this is a causal sequence-regression problem: predictions at a given cycle must use only the measurements available up to that cycle, without relying on future sensor readings. This report evaluates two causal deep learning pipelines on the NASA CMAPSS benchmark, which contains simulated run-to-failure turbofan engine trajectories with operating settings and multivariate sensor measurements.

The first pipeline uses a dilated causal convolutional neural network (CNN), while the second uses a causal Transformer with masked self-attention. Both pipelines share the same preprocessing and evaluation protocol: train-set-only standardization, first- and second-order temporal differences, fixed-length window construction, final-window inference, and a time-weighted regression objective that emphasizes later cycles where degradation evidence is stronger. This shared setup allows the comparison to focus on the architectural difference between convolutional temporal modeling and attention-based temporal modeling.

We compare 50 sampled configurations for each model family using RMSE, MAE, and the NASA asymmetric score. The best recorded CNN configuration obtains a test RMSE of 1.0583, MAE of 0.0741, and NASA score of 10.6022. The best recorded Transformer configuration obtains a test RMSE of 1.7273, MAE of 0.1495, and NASA score of 27.9299. Across the saved experiments, longer input windows consistently improve performance, and compact causal models are more reliable than larger configurations under the implemented search.

The strongest supported finding is not that one architecture is universally superior, but that sufficient temporal context and careful causal evaluation are essential for reliable RUL estimation. The report therefore emphasizes reproducibility, sensitivity analysis, and artifact-backed limitations rather than unsupported ablation claims.

Statement of Originality

This project develops and evaluates data-driven Remaining Useful Life (RUL) estimation pipelines on the NASA CMAPSS dataset. The preprocessing, model formulations, objective functions, and reported results are derived from the implemented convolutional and Transformer notebooks and their recorded experiment outputs.

Contents

1 Introduction **5**
1.1 Motivation . . . 5
1.2 Problem Statement . . . 5
1.3 Contributions . . . 5

2 Related Work **6**
2.1 Classical Prognostics Methods . . . 6
2.2 Deep Sequence Models for RUL . . . 6
2.3 Causal CNNs and Transformers for Time Series . . . 6

3 Dataset Description **7**
3.1 CMAPSS Overview . . . 7
3.2 Train/Test Split and Labels . . . 7
3.3 Sensors and Operating Settings . . . 8
3.4 Derivative Features . . . 8
3.5 Preprocessing Pipeline . . . 8

4 Problem Formulation **9**

5 Methodology **10**
5.1 Dataset Representation and Preprocessing . . . 10
5.2 RUL Target and Window Construction . . . 11
5.3 Dilated Causal Convolutional Model . . . 11
5.4 Causal Transformer Model . . . 12
5.5 Training Objective and Evaluation Metrics . . . 13
5.6 Computational Considerations . . . 14

6 Experimental Setup **15**

7 Results **16**
7.1 Overall Comparison . . . 16
7.2 Best Runs by Model Family . . . 16
7.3 Sensitivity Analysis . . . 18
7.4 Error Behavior and Training Curves . . . 19

8 Controlled Ablation Limitations **23**

9 Discussion **23**

10 Limitations **24**

11 Conclusion **24**

12 Contributions **24**

13 Implementation links **24**

1 Introduction

1.1 Motivation

Modern aviation systems depend on reliable turbofan engines whose degradation is rarely instantaneous. Instead, faults typically emerge through gradual changes in temperature, pressure, speed, flow, and other sensor measurements. Predictive maintenance aims to detect these degradation patterns before failure, allowing maintenance actions to be scheduled based on the observed health state of the engine rather than only on fixed service intervals. Accurate Remaining Useful Life (RUL) estimation is therefore directly connected to safety, operational availability, and maintenance cost.

Traditional maintenance strategies, including scheduled servicing and rule-based monitoring, are limited when degradation signatures are weak, nonlinear, or distributed across many sensors. A rule that works for one operating regime may fail under another, and a single sensor may not reveal the full degradation state. Data-driven sequence models are attractive because they can learn multivariate temporal patterns directly from run-to-failure histories.

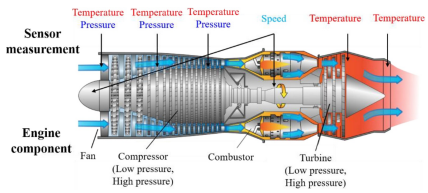


Figure 1: An example terbofan engine

1.2 Problem Statement

The NASA Commercial Modular Aero-Propulsion System Simulation (CMAPSS) dataset provides a benchmark for studying turbofan degradation. It contains multiple engine trajectories with operating settings and sensor readings recorded over time until failure or until a final observed test cycle. In this setting, Remaining Useful Life means the number of cycles left before failure. The learning problem is to estimate the RUL at the final observed cycle of each test engine using only the current and past measurements.

This causality requirement is essential. A deployable prognostics model cannot use measurements from future cycles when estimating the current engine health state. The model must therefore represent temporal dependencies while preventing information leakage from later timesteps. CMAPSS is challenging because degradation is multivariate, nonstationary, and affected by operating conditions; moreover, early cycles often look healthy and carry limited information about the exact failure time.

1.3 Contributions

This project makes four focused contributions:

1. It formulates CMAPSS RUL prediction as a causal multivariate time-series regression task with final-window inference.
2. It implements a reproducible preprocessing pipeline using engine-level splitting, train-only standardization, and temporal derivative features.
3. It compares a dilated causal CNN and a causal Transformer under the same target construction, loss, metrics, and model-selection protocol.
4. It reports sensitivity patterns from 100 saved runs and explicitly separates supported conclusions from ablations that were not run.

2 Related Work

2.1 Classical Prognostics Methods

Predictive maintenance and RUL estimation have long been studied using statistical, physics-based, and classical machine learning methods. Statistical approaches such as proportional hazards models and state-space filtering are useful when degradation dynamics can be described with relatively strong assumptions. Classical machine learning methods, including support vector machines, random forests, and boosting models, can relax some of these assumptions by learning nonlinear mappings from engineered features to health indicators or RUL estimates.

The limitation of many classical methods is that they often require manual feature design and may not naturally model long temporal dependencies. In turbofan degradation, useful information may appear as a gradual trend rather than as a single abnormal sensor value. This motivates sequence models that operate directly on multivariate histories.

2.2 Deep Sequence Models for RUL

Deep learning methods have shown strong performance in modeling degradation trajectories. Recurrent neural networks, especially Long Short-Term Memory (LSTM) networks, have been widely used for RUL prediction because they can retain information over time [3]. Gated recurrent units offer a lighter recurrent alternative. Reconstruction-based sequence models, such as LSTM encoder-decoders, have also been used for anomaly detection by learning representations of normal behavior and measuring reconstruction deviations [4].

The NASA CMAPSS dataset has become one of the most common benchmarks for these methods. Saxena and Goebel introduced the simulator-based run-to-failure data used in many prognostics studies [1]. CMAPSS-specific work often differs in target capping, selected sensors, windowing strategy, model architecture, and evaluation protocol, which makes careful reporting of preprocessing and inference assumptions important.

2.3 Causal CNNs and Transformers for Time Series

Temporal convolutional networks and attention-based sequence models are attractive alternatives to recurrent models. Dilated causal convolutions can efficiently expand receptive field while preserving chronological order [5]. They are computationally efficient because convolutions parallelize across timesteps and can capture local degradation motifs with relatively few layers.

Transformers replace recurrence with self-attention, allowing each timestep to directly attend to earlier timesteps under a causal mask [2]. This is appealing for degradation modeling because long-range dependencies and sensor interactions can be represented without forcing information through a recurrent hidden state. The gap addressed in this project is a direct notebook-backed comparison of a causal CNN and a causal Transformer under the same CMAPSS preprocessing, target construction, loss, and evaluation metrics.

3 Dataset Description

3.1 CMAPSS Overview

CMAPSS consists of simulated turbofan engine degradation trajectories. Each record contains an engine identifier, cycle index, three operating settings, and 21 sensor measurements. The four standard subsets differ in operating conditions and fault modes. The implemented notebooks load every available subset in sorted order from the local `CMaps/` directory and merge them after offsetting engine identifiers to avoid collisions.

Table 1: CMAPSS subset statistics computed from the local raw files. Length values are engine-level cycle counts.

Subset	Train eng.	Test eng.	Train rows	Test rows	RUL labels	Train len. min/med/max	Test len. min/med/max	Condition/fault setting
FD001	100	100	20,631	13,096	100	128/199/362	31/133.5/303	One condition, one fault mode
FD002	260	259	53,759	33,991	259	128/199/378	21/132/367	Multiple conditions, one fault mode
FD003	100	100	24,720	16,596	100	145/220.5/525	38/148/475	One condition, two fault modes
FD004	249	248	61,249	41,214	248	128/234/543	19/153.5/486	Multiple conditions, two fault modes

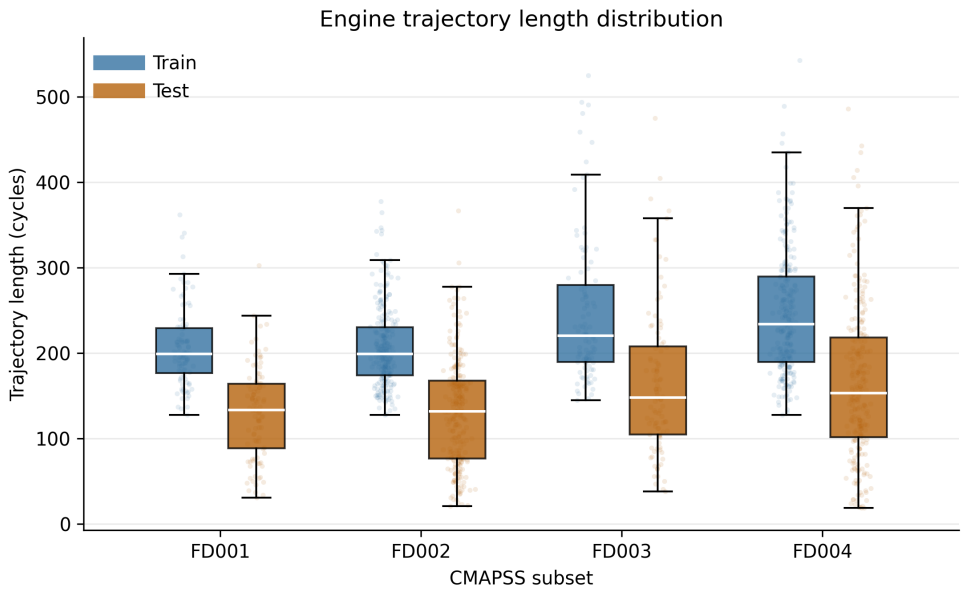


Figure 2: Distribution of engine trajectory lengths across CMAPSS subsets. The wide range of sequence lengths motivates final-window evaluation and sensitivity analysis over window size.

3.2 Train/Test Split and Labels

Training trajectories run until failure, so the RUL at cycle t is computed from the final cycle of the same engine. For example, if an engine fails at cycle 200, then its RUL at cycle 150 is 50 cycles. Test trajectories are different: they are stopped before failure, and the CMAPSS label files provide the true RUL only at the final observed test cycle. The notebooks reconstruct a per-cycle test target by adding the distance from each earlier cycle to the final observed test cycle. Both training and test targets are capped at $R_{\max} = 2000$ in the implemented configuration; for the local CMAPSS files used here, this cap is above the observed trajectory lengths and therefore does not create a restrictive artificial ceiling.

3.3 Sensors and Operating Settings

The notebooks use all three operating settings and all 21 sensor channels as input features. No sensor-dropping rule is applied in the implemented pipeline. Instead, nonstationarity and degradation trends are handled through train-only standardization and derivative feature augmentation. Retaining all channels keeps the comparison architecture-focused: any performance difference is due to the model family and sampled hyperparameters rather than a different feature subset.

Individual sensors do not all degrade monotonically, and some channels remain relatively flat for long periods. The useful signal is therefore multivariate and temporal: the model must combine operating conditions, slowly drifting measurements, and late-life changes rather than relying on one obvious threshold crossing. Figure 3 shows a representative trajectory from the local CMAPSS files.

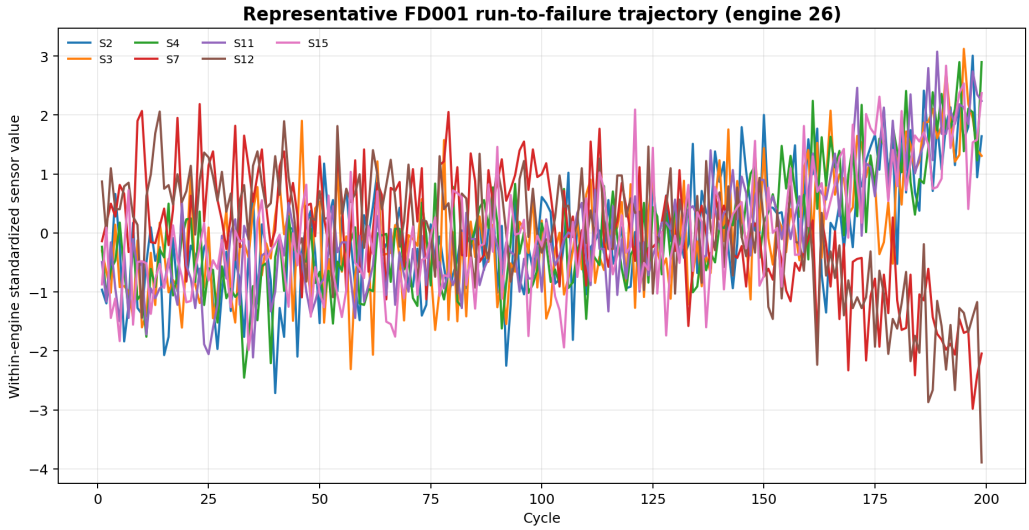


Figure 3: Representative engine trajectory from the local CMAPSS data. Sensor values are standardized within the selected engine only for visualization.

3.4 Derivative Features

First- and second-order differences are included because RUL is tied not only to the current operating state but also to how that state is changing. Velocity features expose local rate of change, while acceleration features expose changes in that rate. These features are motivated by degradation dynamics, but their isolated contribution is not claimed as an ablation result because all saved runs use both derivative channels. Figure 4 illustrates the constructed raw, velocity, and acceleration channels for one representative signal.

3.5 Preprocessing Pipeline

Figure 5 summarizes the shared preprocessing and modeling pipeline. The same normalized and augmented windows are used for both model families, with only the tensor layout and architecture differing between the CNN and Transformer notebooks.

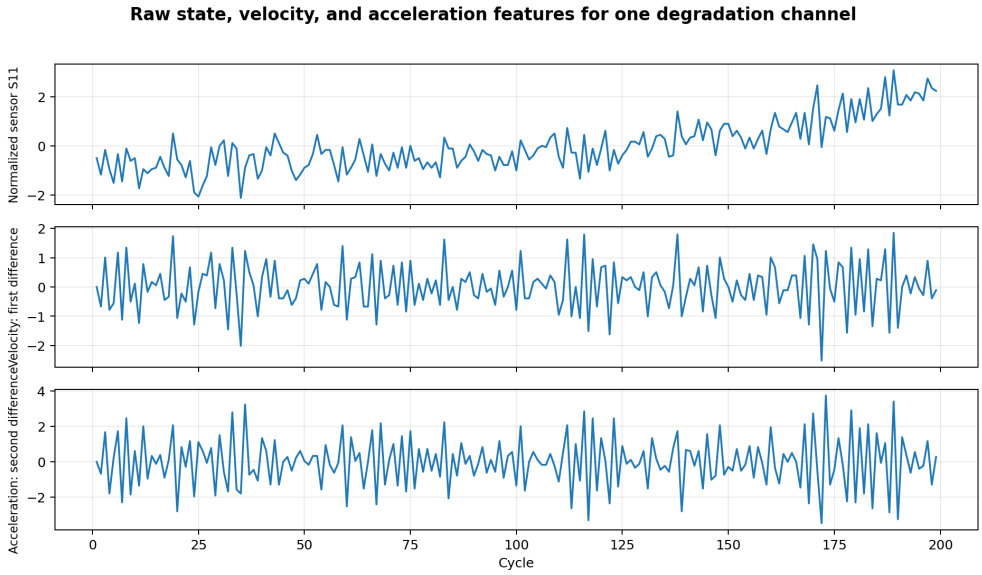


Figure 4: Example of raw state, velocity, and acceleration features used by both model families. This figure illustrates the input representation and is not an ablation result.

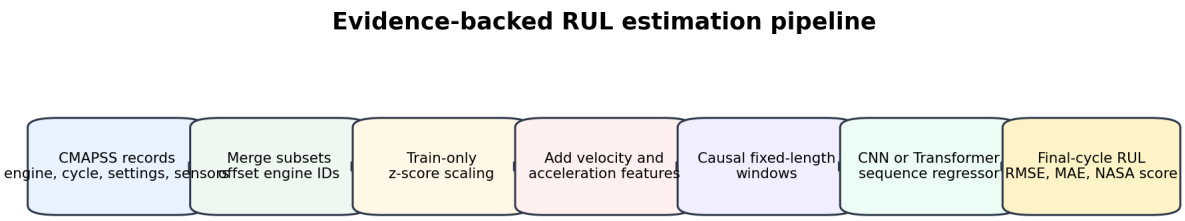


Figure 5: Overall RUL estimation pipeline implemented in the notebooks.

4 Problem Formulation

Given an observed partial trajectory $(\tilde{x}_{i,1}, \ldots, \tilde{x}_{i,K_i})$, where $\tilde{x}_{i,t}$ denotes the processed feature vector at cycle t, the goal is to estimate the remaining useful life at the final observed cycle:

$$f_\theta(\tilde{x}_{i,1}, \ldots, \tilde{x}_{i,K_i}) \rightarrow \widehat{\mathrm{RUL}}_i. \tag{1}$$

The model must satisfy causality: a prediction at cycle t may depend on $\tilde{x}_{i,\tau}$ only for $\tau \leq t$. Operationally, the model is allowed to use the engine history up to the current cycle, but it is not allowed to look at measurements that would only be available in the future. This constraint is enforced by left-padded causal convolutions in the CNN and by an upper-triangular attention mask in the Transformer.

5 Methodology

This section describes the shared CMAPSS data pipeline and the two implemented causal sequence models: a dilated causal convolutional neural network and a causal Transformer. The notation, model components, losses, and metrics follow the corresponding notebooks and their recorded experiment outputs.

5.1 Dataset Representation and Preprocessing

Each CMAPSS engine is represented as an independent multivariate trajectory. Let $i \in \{1, \ldots, N\}$ index engines and let $t \in \{1, \ldots, L_i\}$ denote the operating cycle of engine i. At each cycle, the observation vector is

$$x_{i,t} \in \mathbb{R}^{24}, \tag{2}$$

where the 24 input channels consist of three operational settings and 21 sensor measurements. The full trajectory of engine i is denoted

$$X_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,L_i}). \tag{3}$$

Both notebooks standardize features using statistics computed only from the training split. Standardization puts sensors with different physical units and magnitudes on a comparable scale, while using train-only statistics prevents information from the validation or test engines from leaking into preprocessing. For feature j, the normalized value is

$$\bar{x}_{i,t}^{(j)} = \frac{x_{i,t}^{(j)} - \mu_j}{\sigma_j}, \qquad j = 1, \ldots, 24, \tag{4}$$

where μ_j and σ_j are the training-set mean and standard deviation of feature j. Very small standard deviations are replaced by 1.0 in the implementation to avoid numerical instability.

To expose local degradation dynamics, both models augment the normalized features with first- and second-order temporal differences. The first difference measures how much a signal changes from one cycle to the next, and the second difference measures how that rate of change itself changes. This is useful because degradation may appear as a slow trend even when an individual sensor value is not yet extreme:

$$v_{i,t} = \begin{cases} 0, & t = 1, \\ \bar{x}_{i,t} - \bar{x}_{i,t-1}, & t > 1, \end{cases} \tag{5}$$

and

$$a_{i,t} = \begin{cases} 0, & t = 1, \\ v_{i,t} - v_{i,t-1}, & t > 1. \end{cases} \tag{6}$$

This matches the notebooks' use of `np.diff(..., prepend=first)`, which makes the first difference vector zero at the start of each engine sequence. The final model input at each cycle is

$$\tilde{x}_{i,t} = [\bar{x}_{i,t}; v_{i,t}; a_{i,t}]. \tag{7}$$

Since both velocity and acceleration augmentation are enabled in all sampled experiments, this representation supplies the models with three views of the same engine state: the current normalized measurements, their local rate of change, and the change in that rate.

5.2 RUL Target and Window Construction

For training trajectories, the RUL target is computed from the final observed training cycle of each engine and capped at $R_{\max} = 2000$:

$$y_{i,t} = \min(L_i - t, R_{\max}). \tag{8}$$

For a test engine, the true remaining life after the last observed cycle is supplied by the CMAPSS test labels. If K_i is the last observed cycle and R_i^{test} is the label for the final observed cycle, the per-cycle target used internally is

$$y_{i,t} = \min\left(R_i^{\text{test}} + (K_i - t), R_{\max}\right). \tag{9}$$

Training samples are fixed-length windows. A window is a short segment of an engine's history that the model receives as one input sequence. For a window length T and stride s, a training sample beginning at cycle r is

$$W_{i,r} = (\tilde{x}_{i,r}, \tilde{x}_{i,r+1}, \ldots, \tilde{x}_{i,r+T-1}), \tag{10}$$

with the corresponding RUL sequence

$$Y_{i,r} = (y_{i,r}, y_{i,r+1}, \ldots, y_{i,r+T-1}). \tag{11}$$

Training windows are generated with the sampled stride, while validation and test evaluation use the final available window for each engine. This mirrors deployment: when an engine is inspected at the current cycle, the model should use the most recent available history and return the RUL estimate for that current cycle.

5.3 Dilated Causal Convolutional Model

The convolutional notebook implements a stack of causal one-dimensional convolutions over time. The input tensor is arranged as (B, C, T), where B is batch size, C is the augmented feature dimension, and T is the window length. For layer $\ell \in \{0, \ldots, L-1\}$, the dilation is

$$d_\ell = 2^\ell. \tag{12}$$

The convolution uses left-only padding of $(k-1)d_\ell$ samples for kernel size k, so the output at time t depends only on current and past values. Dilation skips over timesteps in a regular pattern, which lets deeper layers cover a longer history without making the network very deep. In mathematical form, the hidden state of a causal convolutional layer can be written as

$$h_t^{(\ell)} = \phi\left(\sum_{m=0}^{k-1} W_{\ell,m} h_{t-md_\ell}^{(\ell-1)} + b_\ell\right), \tag{13}$$

where unavailable past values are supplied by left padding and ϕ is the GELU nonlinearity used in the notebook. Each convolutional block applies batch normalization before convolution, followed by GELU and dropout.

The effective receptive field of the convolutional stack is

$$R_{\text{CNN}} = 1 + \sum_{\ell=0}^{L-1} (k-1)2^\ell. \tag{14}$$

Predictions before this receptive field is fully populated are partially influenced by padding; therefore, the convolutional loss masks the first $R_{\text{CNN}} - 1$ positions in each window. This avoids training the model on early outputs that have not yet seen enough real history. After the convolutional backbone, the hidden

representation is transposed to (B, T, H) and passed through a regression head:

$$\hat{y}_{i,t}^{\text{CNN}} = g_{\text{CNN}}(h_{i,t}^{(L)}). \tag{15}$$

The head consists of layer normalization, a linear projection to half the hidden width, GELU, dropout, and a final linear projection to one scalar RUL value.

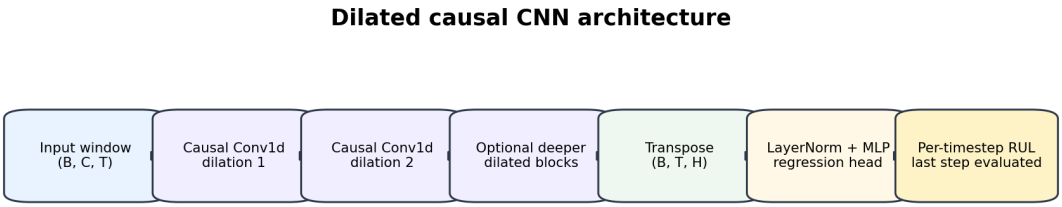


Figure 6: Causal CNN architecture implemented in the convolution notebook.

5.4 Causal Transformer Model

The Transformer notebook implements a causal self-attention model over the same augmented sequences, using tensors arranged as (B, T, C). Unlike the CNN, which combines information through local convolutional filters, the Transformer can directly compare one timestep with earlier timesteps through attention. Each input vector is first linearly projected into a latent space:

$$z_{i,t}^{(0)} = W_e \tilde{x}_{i,t} + b_e. \tag{16}$$

Fixed sinusoidal positional encodings are added to preserve temporal order:

$$h_{i,t}^{(0)} = z_{i,t}^{(0)} + p_t. \tag{17}$$

Each Transformer layer applies masked multi-head self-attention followed by a position-wise feed-forward block. For one attention head, queries, keys, and values are

$$Q = HW_Q, \qquad K = HW_K, \qquad V = HW_V. \tag{18}$$

Causal attention is computed as

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \tag{19}$$

with the upper-triangular mask

$$M_{t,\tau} = \begin{cases} 0, & \tau \le t, \\ -\infty, & \tau > t. \end{cases} \tag{20}$$

This prevents the prediction at cycle t from attending to future cycles in the same window. Entries above the diagonal are assigned $-\infty$ before the softmax, so their attention probability becomes zero. The implementation uses PyTorch's `TransformerEncoder` with this causal mask, batch-first tensors, pre-normalized Transformer blocks, GELU activation, dropout, and a feed-forward dimension of $4d_{\text{model}}$.

After L Transformer layers, a regression head maps each hidden state to a scalar prediction:

$$\hat{y}_{i,t}^{\mathrm{Tr}} = g_{\mathrm{Tr}}(h_{i,t}^{(L)}). \tag{21}$$

The head consists of layer normalization, a linear projection to $d_{\mathrm{model}}/2$, GELU, dropout, and a final linear projection. Since every Transformer position can attend to all previous positions immediately through the causal mask, the notebook sets the Transformer receptive-field warmup parameter to 1.

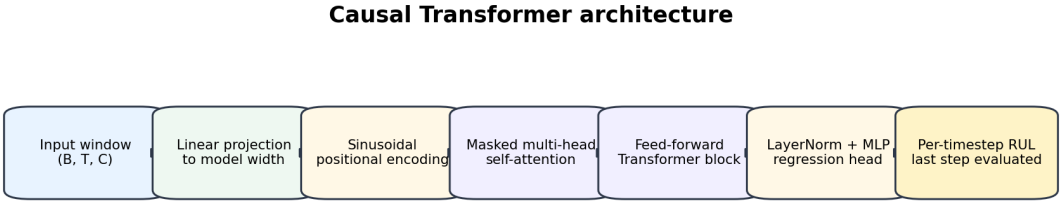


Figure 7: Causal Transformer architecture implemented in the Transformer notebook.

Both architectures are causal, but they impose different inductive biases. The CNN expands a local receptive field through dilation and is computationally linear in window length for fixed kernel size, while the Transformer allows each timestep to attend to all earlier timesteps at quadratic attention cost.

5.5 Training Objective and Evaluation Metrics

Both notebooks train with a masked, time-weighted mean squared error. The mask removes padded or warmup positions from the loss, and the time weighting gives more importance to later positions in the window. This matches the final-window evaluation setting, where the last prediction is the one used for test RUL. For a batch of padded windows, let $m_{b,t} \in \{0, 1\}$ indicate whether timestep t is valid for batch element b. Later timesteps are weighted more heavily using

$$w_t = \exp\left(\beta \frac{t}{T-1}\right), \qquad t = 0, \ldots, T-1. \tag{22}$$

Let r denote the model-specific warmup region: $r = R_{\mathrm{CNN}}$ for the convolutional model and $r = 1$ for the Transformer. The valid mask used in the loss is

$$\tilde{m}_{b,t} = \begin{cases} 0, & t < r-1, \\ m_{b,t}, & t \geq r-1. \end{cases} \tag{23}$$

The resulting objective is

$$\mathcal{L} = \frac{\sum_b \sum_{t=0}^{T-1} \tilde{m}_{b,t} w_t \left(\hat{y}_{b,t} - y_{b,t}\right)^2}{\sum_b \sum_{t=0}^{T-1} \tilde{m}_{b,t} w_t}. \tag{24}$$

This objective reflects the structure of RUL prediction: early-cycle observations often contain limited degradation evidence, whereas later-cycle observations are more informative for final-cycle prognostics.

At inference time, each model processes the final available window of a test engine and only the last prediction is evaluated:

$$\widehat{\mathrm{RUL}}_i = \hat{y}_{i,K_i}. \tag{25}$$

Performance is measured with RMSE, MAE, and the NASA asymmetric score. RMSE emphasizes large errors because the error is squared before averaging, while MAE gives the average absolute error in a

more direct way. Both are symmetric, meaning overestimation and underestimation of the same size are treated equally:

$$\mathrm{RMSE} = \sqrt{\frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \left(\widehat{\mathrm{RUL}}_i - \mathrm{RUL}_i\right)^2}, \tag{26}$$

$$\mathrm{MAE} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \left|\widehat{\mathrm{RUL}}_i - \mathrm{RUL}_i\right|, \tag{27}$$

The NASA score is different because predictive maintenance has an asymmetric risk profile. Predicting too much remaining life can delay maintenance beyond a safe point, so overestimation is penalized more strongly. With $d_i = \widehat{\mathrm{RUL}}_i - \mathrm{RUL}_i$,

$$S_i = \begin{cases} \exp\left(-\dfrac{d_i}{13}\right) - 1, & d_i < 0, \\ \exp\left(\dfrac{d_i}{10}\right) - 1, & d_i \geq 0, \end{cases} \qquad S = \sum_{i=1}^{N_{\text{test}}} S_i. \tag{28}$$

5.6 Computational Considerations

The causal CNN has computational cost that grows approximately linearly with sequence length for a fixed kernel size and hidden width. Dilation expands temporal coverage without requiring every timestep to interact with every other timestep, making the convolutional model relatively efficient for long windows. The Transformer has a stronger long-range modeling mechanism because each timestep can attend to all earlier timesteps, but self-attention requires an attention matrix over the window and therefore scales quadratically with window length. This creates a trade-off: the CNN is more computationally economical for long histories, while the Transformer offers a more flexible global dependency mechanism at higher memory cost.

Table 2: Qualitative complexity comparison for a window of length T, hidden width H, kernel size k, and L layers.

Model	**Dominant temporal cost**	**Practical implication**
Causal CNN	$O(LTkH^2)$	Linear in window length for fixed kernel size
Causal Transformer	$O(LT^2H + LTH^2)$	Quadratic attention memory and compute in T

Algorithm 1 Training and final-window inference protocol

```
Load all CMAPSS subsets and offset engine identifiers across subsets.
Split training engines into train and validation groups at the engine level.
Compute feature means and standard deviations from the training group only.
Normalize features, add velocity and acceleration channels, and construct causal windows.
for each sampled configuration do
    Train the causal CNN or Transformer with masked time-weighted MSE.
    Evaluate validation engines using their final available windows.
    Save the checkpoint with the best validation RMSE.
end for
Evaluate the selected checkpoint on each test engine using only its final available window.
```

6 Experimental Setup

Both pipelines were implemented in PyTorch and executed on the merged CMAPSS subsets discovered in the local `CMaps/` directory. Engine identifiers were offset across subsets to prevent collisions. The training engines were split into training and validation groups at the engine level using an 80/20 split, so windows from the same engine were not mixed across the two splits. Feature standardization was fit only on the training engines for each run. Model selection used the checkpoint with the best validation RMSE, and test evaluation used the final available window for each test engine.

Each model family ran 50 sampled experiments for 200 epochs with the Adam optimizer. The sampling seed schedule was $42 + i$ for experiment index i, giving seeds 42 through 91. Both searches used $R_{\max} = 2000$ and enabled velocity and acceleration augmentation. The saved CNN artifacts are under `runs_cmapss/`, with aggregate results in `runs_cmapss/all_results.csv` and the best checkpoint in `runs_cmapss/exp_046/best_model.pt`. The Transformer artifacts are under `runs_cmapss_Tranformers/`, with aggregate results in `runs_cmapss_Tranformers/all_results.csv` and the best checkpoint in `runs_cmapss_Tranformers/exp_016/best_model.pt`. Table 3 summarizes the sampled hyperparameter ranges.

Table 3: Hyperparameter search spaces used in the convolutional and Transformer notebooks.

Hyperparameter	**Causal CNN values**	**Transformer values**
Window size	200, 500, 800, 1000	200, 500, 800, 1000
Stride	1, 2, 5	50, 60, 70, 80
Batch size	32, 64, 128, 256	32, 64, 128, 256
Learning rate	10^{-4}, 3×10^{-4}, 5×10^{-4}, 8×10^{-4}, 10^{-3}	Same
Weight decay	0, 10^{-5}, 10^{-4}, 10^{-3}	Same
Time-weight exponent β	0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0	Same
Hidden width	32, 64, 96, 128	32, 64, 96, 128
Layers	2, 3, 4, 5	2, 3, 4, 5
Kernel size	3, 5, 7	–
Attention heads	–	2, 4, 8
Dropout	0.2, 0.3, 0.5, 0.7, 0.8	0.2, 0.3, 0.5, 0.7, 0.8

7 Results

7.1 Overall Comparison

Table 4 compares the best recorded run from each model family, ranked by NASA score. For all three metrics, lower values are better. The causal CNN achieves the strongest overall result, with lower RMSE, lower MAE, and a lower NASA score than the best Transformer run. The result is consistent across both symmetric error metrics and the asymmetric NASA score, so the CNN is not merely improving one metric at the expense of another.

Table 4: Best recorded configuration from each model family.

Model	Best experiment	Window	Test RMSE	Test MAE	NASA score
Causal CNN	exp_046	1000	1.0583	0.0741	10.6022
Transformer	exp_016	1000	1.7273	0.1495	27.9299

The comparison in Table 4 is based on the aggregate metrics saved for each completed experiment. Since per-engine prediction traces were not exported during these runs, the results are interpreted through aggregate error metrics and search-level sensitivity plots rather than individual engine case studies. This keeps the claims aligned with the saved artifacts: the report can compare runs and hyperparameter trends, but it cannot identify the exact hardest individual engines without rerunning inference and saving per-engine predictions.

Table 5: Best and median test performance across the 50 recorded runs per model family. Medians summarize search stability and are not model-selection criteria.

Model	Best RMSE	Median RMSE	Best NASA	Median NASA
Causal CNN	1.0583	9.0728	10.6022	1.29×10^6
Transformer	1.7273	13.2728	27.9299	1.30×10^6

7.2 Best Runs by Model Family

Table 6 reports the five best causal CNN configurations. The best model used a window size of 1000, stride 2, batch size 256, learning rate 5×10^{-4}, no weight decay, $\beta = 2.0$, hidden width 32, two dilated convolutional layers, kernel size 3, and dropout 0.5. Its validation RMSE was 0.0143 and its test NASA score was 10.6022.

Table 6: Top causal CNN runs ranked by NASA score. Results are taken from the saved CNN run CSV.

Exp.	Win.	Stride	Batch	LR	WD	β	Hidden	Layers	Kernel	Drop.	Val RMSE	Test RMSE	Test MAE	NASA
exp_046	1000	2	256	0.0005	0.0000	2.00	32	2	3	0.5	0.0143	1.0583	0.0741	10.6022
exp_016	800	2	256	0.0008	0.0000	0.50	32	5	3	0.5	0.0527	1.7350	0.1545	27.7651
exp_007	1000	2	256	0.0003	0.0001	0.75	64	4	5	0.8	0.0646	1.7334	0.1664	28.2897
exp_008	500	1	256	0.0001	0.0000	0.50	64	3	7	0.7	0.0782	1.7382	0.1776	29.0770
exp_009	1000	5	256	0.0010	0.0001	0.25	32	4	3	0.5	0.1395	1.6962	0.2198	29.9642

Table 7 reports the five best Transformer configurations. The best Transformer used a window size of 1000, stride 70, batch size 256, learning rate 8×10^{-4}, weight decay 10^{-3}, $\beta = 0.25$, $d_{\text{model}} = 32$, two Transformer layers, four attention heads, and dropout 0.5.

Figure 8 compares the ten best saved configurations from each model family under the NASA score.

Table 7: Top Transformer runs ranked by NASA score. Results are taken from the saved Transformer run CSV.

Exp.	Win.	Stride	Batch	LR	WD	β	d_{model}	Layers	Heads	Drop.	Val RMSE	Test RMSE	Test MAE	NASA
exp_016	1000	70	256	0.0008	0.0010	0.25	32	2	4	0.5	0.0136	1.7273	0.1495	27.9299
exp_039	500	70	256	0.0003	0.0001	2.00	32	4	2	0.5	0.0244	1.7560	0.1858	30.2536
exp_010	1000	60	256	0.0001	0.0001	0.75	32	4	8	0.8	0.2544	1.7382	0.1975	31.5440
exp_049	1000	70	256	0.0003	0.0010	0.75	96	5	4	0.7	0.2094	1.7568	0.2628	35.5437
exp_009	1000	80	256	0.0010	0.0001	0.25	32	4	8	0.5	0.3406	1.7839	0.4173	42.9590

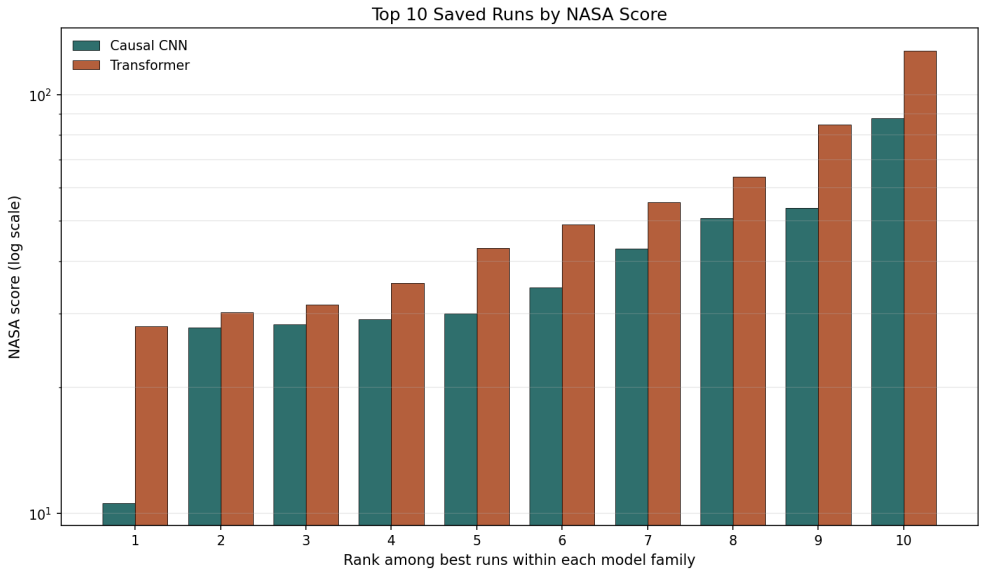


Figure 8: Top ten saved CNN and Transformer configurations ranked by NASA score. A logarithmic axis is used because the NASA score grows exponentially for large errors.

7.3 Sensitivity Analysis

Figure 9 summarizes the distribution of test RMSE values across the 50 sampled configurations per model family. The spread is large for both searches, which means the reported best models should be interpreted as selected configurations from a stochastic hyperparameter search rather than as inevitable outcomes of the architecture alone. Table 5 reinforces the same point: the best runs are much stronger than the median runs, so hyperparameter choice materially affects performance. In other words, the experiment supports a comparison of the best recorded pipelines, but it also shows that poor settings can make either architecture fail.

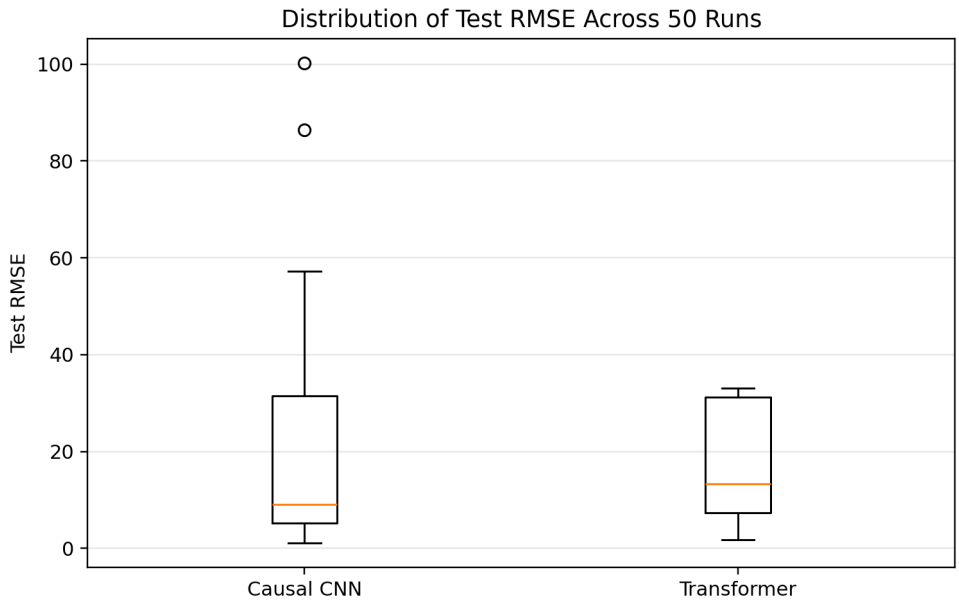


Figure 9: Distribution of test RMSE values across the 50 sampled configurations for each model family.

Window length is the clearest sensitivity pattern in the saved results. For the CNN, the median RMSE improves from 33.5079 at window size 200 to 3.5596 at window size 1000. For the Transformer, the median RMSE improves from 32.7237 at window size 200 to 2.6804 at window size 1000. Figure 10 shows the median and interquartile spread at each window size, with individual runs shown as faint points. Longer windows are useful because late-cycle RUL estimation benefits from a longer observed degradation history, especially when the model uses derivative features.

Figure 11 shows the same window-size pattern under the asymmetric NASA score.

Figure 12 summarizes the univariate spread in median test RMSE across common hyperparameters. This plot is not a controlled ablation because the configurations were sampled jointly, but it gives a compact view of which choices varied most strongly in the recorded search. Window size, batch size, dropout, and the time-weighting exponent all show substantial variation, while the best runs for both model families use compact hidden widths.

The aggregate results do not justify a strong claim that any single hyperparameter value is universally optimal. The more defensible interpretation is that compact models can work well in this pipeline when paired with sufficient temporal context and stable optimization settings.

Figure 13 compares validation RMSE and test RMSE across all saved runs. The CNN shows a strong validation–test relationship, with correlation approximately 0.84. The Transformer shows a much weaker relationship, with correlation approximately 0.04. This suggests that validation RMSE was a more reliable model-selection signal for the CNN search than for the Transformer search under the implemented split and hyperparameter ranges.

Figure 14 compares RMSE and NASA score. The relationship is positive but not linear because the

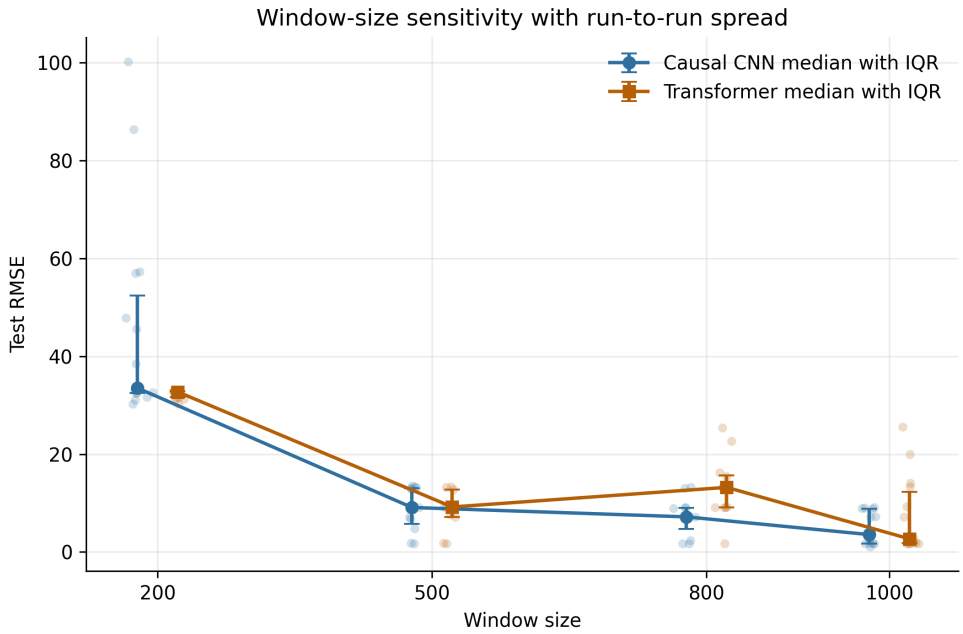


Figure 10: Sensitivity of test RMSE to window size across sampled CNN and Transformer configurations. Markers show medians, bars show interquartile ranges, and faint points show individual runs.

NASA score is asymmetric and exponential. Configurations with moderate RMSE can still receive very large NASA scores if their errors include risky overestimation of remaining life.

7.4 Error Behavior and Training Curves

The best CNN has the lowest NASA score, which is important because the NASA metric penalizes RUL overestimation more strongly than underestimation. In maintenance terms, overestimation is more dangerous: a model that predicts too much remaining life can delay maintenance beyond the true failure point. Underestimation is still undesirable because it can cause early maintenance, but it is less severe in the NASA scoring function.

The worst CNN by NASA score is exp_037, with test RMSE 38.4943, MAE 29.1333, and NASA score approximately 3.41×10^{11}. The worst CNN by RMSE is exp_027, with RMSE 100.2669 and MAE 89.6458. For the Transformer, exp_048 is worst by both RMSE and NASA score, with RMSE 33.0754, MAE 12.7370, and NASA score approximately 3.29×10^{6}. These failed configurations are concentrated at short windows, especially window size 200, supporting the interpretation that insufficient temporal context can lead to unsafe RUL behavior.

Per-engine prediction traces were not exported during the recorded experiments, so the error analysis is restricted to aggregate behavior supported by the saved metrics. Figure 15 includes the saved training-history plots for the best CNN and Transformer runs, providing a qualitative view of convergence and validation behavior.

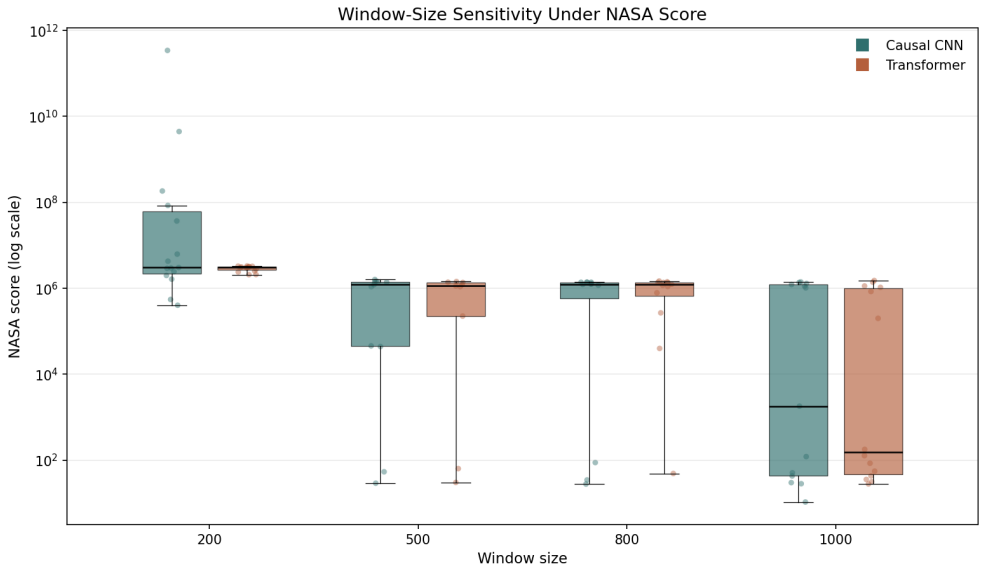


Figure 11: Sensitivity of NASA score to window size across sampled CNN and Transformer configurations. The logarithmic scale reflects the large penalty assigned to unsafe large errors.

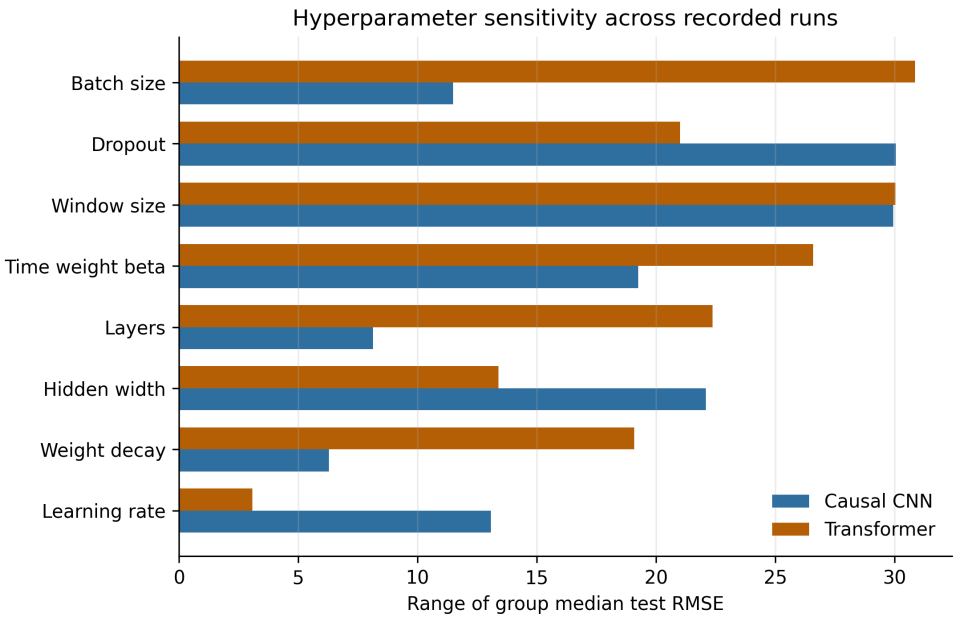


Figure 12: Univariate sensitivity summary across the recorded hyperparameter search. Bars show the range of group median test RMSE across values of each hyperparameter.

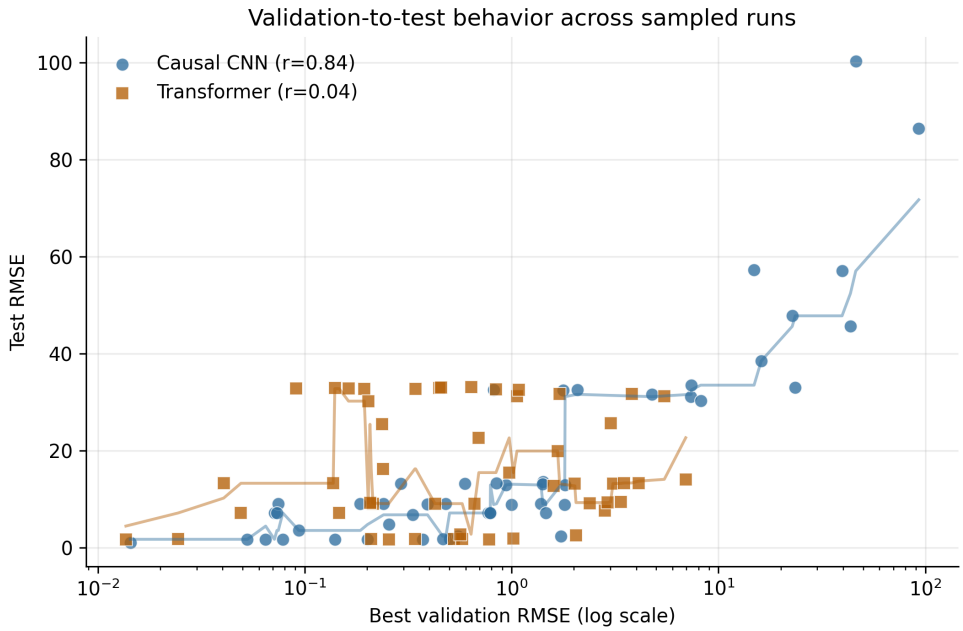


Figure 13: Validation RMSE versus test RMSE across the 50 sampled configurations for each model family.

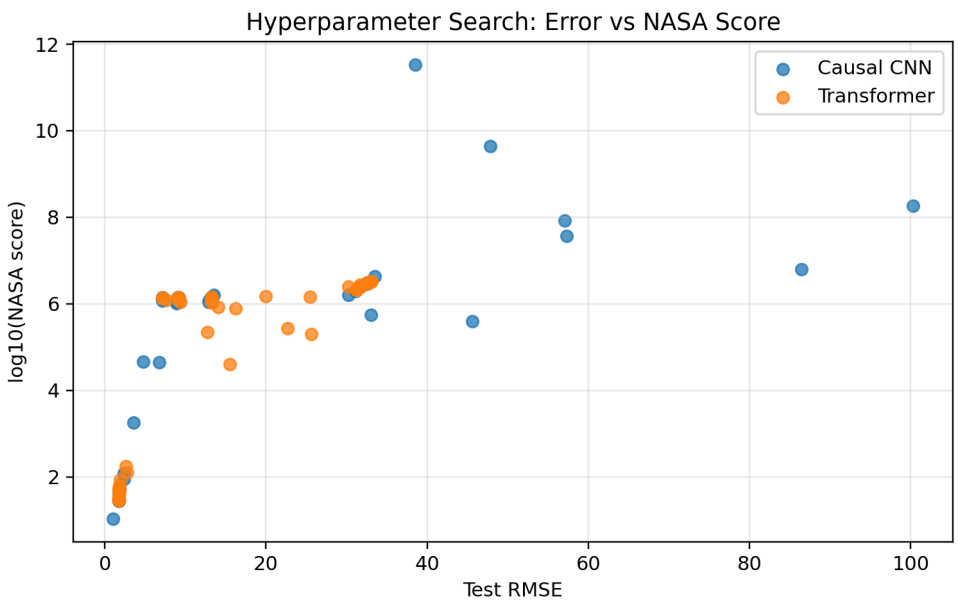
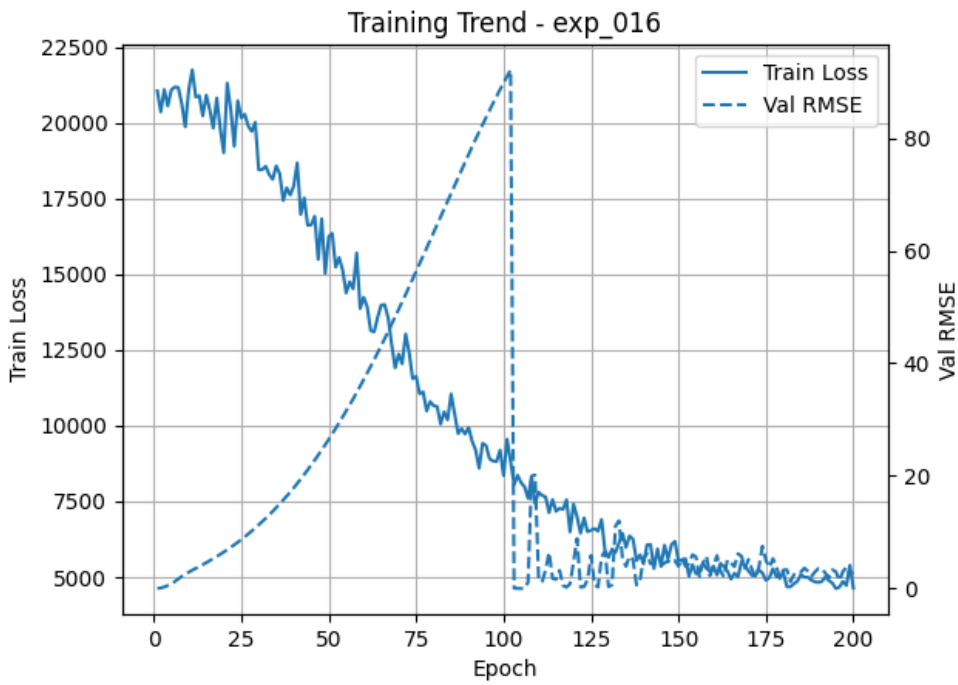


Figure 14: Relationship between RMSE and NASA asymmetric score across sampled configurations.

(a) Causal CNN training history.



(b) Transformer training history.

Figure 15: Training loss and validation RMSE history for the best recorded CNN and Transformer configurations.

8 Controlled Ablation Limitations

The available saved experiments support sensitivity analysis rather than a complete controlled ablation study. All 100 recorded runs use both velocity and acceleration features, and all sampled configurations use a positive time-weighting exponent β. Therefore, the report does not claim a raw-only, velocity-only, acceleration-only, or no-time-weight ablation. The defensible conclusion is narrower: within the implemented feature pipeline, performance is highly sensitive to window size, model family, optimization settings, and model selection.

The strongest evidence from the current artifacts is that longer windows consistently help both architectures, and that the best CNN combines a long window with a compact convolutional backbone. The derivative features and time-weighted loss remain theoretically motivated and consistently present in the implementation, but their isolated contributions would require additional controlled runs that disable each component one at a time.

9 Discussion

The implemented pipelines treat RUL estimation as causal sequence regression. This is essential for predictive maintenance because a deployable model cannot use future measurements when estimating the current health state. The CNN enforces causality through left-padded dilated convolutions, while the Transformer enforces causality through an upper-triangular attention mask. Both models therefore produce predictions that depend only on current and previous cycles.

The shared time-weighted objective is consistent with degradation physics. Early engine cycles often resemble healthy operation and may carry limited information about exact failure time. Later cycles contain stronger degradation signatures, so increasing the regression weight across the window encourages the models to focus on the portion of the trajectory most relevant to final RUL estimation. The CNN additionally masks early predictions that are still within the convolutional warmup region, avoiding loss contributions from outputs influenced by padding.

The best recorded CNN configuration outperforms the best recorded Transformer configuration on RMSE, MAE, and NASA score. This should be read as an artifact-backed result for the implemented search, not as a general proof that causal CNNs dominate Transformers for RUL estimation. The saved runs show that a compact dilated convolution was easier to select reliably under the current pipeline, while the Transformer search had weaker validation-to-test alignment.

The results also show why the NASA score should be interpreted alongside RMSE and MAE. RMSE measures average squared error, while the NASA score encodes a maintenance preference: overestimating RUL is worse than underestimating it. This is why failed runs can produce extremely large NASA scores even when their RMSE values are not the largest in the entire search. For a prognostics pipeline, this safety-oriented asymmetry is not a detail; it changes which model behavior is acceptable.

Exporting per-engine predictions in future runs would enable predicted-versus-true plots, residual histograms, overestimation analysis, and per-subset breakdowns in addition to the aggregate metrics reported here.

10 Limitations

The results come from sampled hyperparameter experiments rather than exhaustive searches. Therefore, the reported best models should be interpreted as the best recorded configurations from the implemented experiments, not as globally optimal architectures. The unusually large NASA scores in some failed runs also indicate that the asymmetric metric can become very large when a model substantially overestimates RUL.

The experimental artifacts do not include controlled raw-only, derivative-only, or no-time-weight runs, so this report cannot isolate the independent causal contribution of each preprocessing component. The saved artifacts also do not include per-engine prediction files, which prevents a fully grounded case-study analysis of the hardest engines without rerunning inference from checkpoints.

Finally, all experiments are conducted on CMAPSS simulation data. Real flight data would introduce maintenance interventions, sensor replacement, calibration drift, missing values, and distribution shifts across aircraft and operating regimes. A deployable system would need validation under these conditions and would likely require uncertainty estimates or conservative decision thresholds in addition to point RUL predictions.

11 Conclusion

This work implements and compares two causal deep learning frameworks for RUL estimation on CMAPSS turbofan degradation trajectories. Both models combine train-only standardization, dynamics-based feature augmentation, causal temporal modeling, time-weighted regression, and final-window inference. The best recorded causal CNN achieves the strongest saved result, with test RMSE 1.0583, MAE 0.0741, and NASA score 10.6022. The best recorded Transformer achieves test RMSE 1.7273, MAE 0.1495, and NASA score 27.9299.

The main empirical pattern is that long causal windows and compact temporal models are effective for this pipeline. Future work should save per-engine predictions, add controlled ablations for raw features, velocity, acceleration, and time weighting, repeat the best configurations across multiple seeds, and report per-subset performance if experiments are rerun.

12 Contributions

- Sai Deepak Sana - Problem Statement finding and Causal CNN Design and Implementation
- Ponguru Bhanu - Core architecture design and causal transformer implementation.
- Peddineni Harshith - Debugging, Validation of results and report writing.

13 Implementation links

- **GitHub**: Project Source Code
- **Kaggle (CNN)**: CNN Model Notebook
- **Kaggle (Transformer)**: Transformer Model Notebook

References

[1] Saxena, A., Goebel, K., Simon, D., and Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 International Conference on Prognostics and Health Management*, pages 1–9. IEEE.

[2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

[3] Zheng, S., Ristovski, K., Farahat, A., and Gupta, C. (2017). Long short-term memory network for remaining useful life estimation. In *2017 IEEE International Conference on Prognostics and Health Management*, pages 88–95.

[4] Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., and Shroff, G. (2016). LSTM-based encoder-decoder for multi-sensor anomaly detection. In *ICML 2016 Anomaly Detection Workshop*.

[5] Bai, S., Kolter, J. Z., and Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*.